1.1 ./Link.Foundation.Links.Notation/FormatConfig.cs

```cs
namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// FormatConfig is an alias for FormatOptions to maintain API consistency
    /// with the Python implementation.
    ///
    /// C# uses 'FormatOptions' internally, but provides 'FormatConfig'
    /// as an alias to match Python's naming convention.
    /// </summary>
    public class FormatConfig : FormatOptions
    {
        // FormatConfig inherits all functionality from FormatOptions
        // No additional implementation needed - this is purely for naming consistency
    }
}
```

1.2 ./Link.Foundation.Links.Notation/FormatOptions.cs

```cs
using System;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// FormatOptions for Lino notation formatting.
    /// Provides configuration options for controlling how Link objects are formatted.
    /// </summary>
    public class FormatOptions
    {
        /// <summary>
        /// If true, omit parentheses where safe (default: false)
        /// </summary>
        public bool LessParentheses { get; set; } = false;

        /// <summary>
        /// Maximum line length before auto-indenting (default: 80)
        /// </summary>
        public int MaxLineLength { get; set; } = 80;

        /// <summary>
        /// If true, indent lines exceeding MaxLineLength (default: false)
        /// </summary>
        public bool IndentLongLines { get; set; } = false;

        /// <summary>
        /// Maximum number of references before auto-indenting (default: null = unlimited)
        /// </summary>
        public int? MaxInlineRefs { get; set; } = null;

        /// <summary>
        /// If true, group consecutive links with same ID (default: false)
        /// </summary>
        public bool GroupConsecutive { get; set; } = false;

        /// <summary>
        /// String to use for indentation (default: "  " = two spaces)
        /// </summary>
        public string IndentString { get; set; } = "  ";

        /// <summary>
        /// If true, prefer inline format when under thresholds (default: true)
        /// </summary>
        public bool PreferInline { get; set; } = true;

        /// <summary>
        /// Check if line should be indented based on length
        /// </summary>
        /// <param name="line">The line to check</param>
        /// <returns>True if line should be indented based on length threshold</returns>
        public bool ShouldIndentByLength(string line)
        {
            if (!IndentLongLines)
            {
                return false;
            }
            // Count printable unicode characters
            return line.Length > MaxLineLength;
        }

```

```
        /// <summary>
        /// Check if link should be indented based on reference count
        /// </summary>
        /// <param name="refCount">Number of references in the link</param>
        /// <returns>True if link should be indented based on reference count threshold</returns>
        public bool ShouldIndentByRefCount(int refCount)
        {
            if (MaxInlineRefs == null)
            {
                return false;
            }
            return refCount > MaxInlineRefs.Value;
        }
    }
}
```

1.3 ./Link.Foundation.Links.Notation/ILinksGroupListExtensions.cs

```
using System.Collections.Generic;
using System.Runtime.CompilerServices;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Provides extension methods for collections of <see cref="LinksGroup{TLinkAddress}"/> instances.
    /// </summary>
    public static class ILinksGroupListExtensions
    {
        /// <summary>
        /// Converts a collection of links groups to a flat list of links by processing each group hierarchically.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
        /// <param name="groups">The collection of links groups to convert.</param>
        /// <returns>A flat list of links representing all the links from the input groups.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static List<Link<TLinkAddress>> ToLinksList<TLinkAddress>(this IList<LinksGroup<TLinkAddress>> groups)
        {
            var list = new List<Link<TLinkAddress>>();
            for (var i = 0; i < groups.Count; i++)
            {
                CollectLinksWithIndentedIdSyntaxSupport(list, groups[i], null);
            }
            return list;
        }

        private static void CollectLinksWithIndentedIdSyntaxSupport<TLinkAddress>(List<Link<TLinkAddress>> list, LinksGroup<TLinkAddress> group, Link<TLinkAddress>? parentDependency)
        {
            var link = group.Link;
            var groups = group.Groups;

            if (groups != null && groups.Count > 0)
            {
                bool isIndentedIdSyntax = link.Id != null &&
                                          (link.Values == null || link.Values.Count == 0);

                if (isIndentedIdSyntax && !parentDependency.HasValue)
                {
                    var childValues = new List<Link<TLinkAddress>>();
                    for (int i = 0; i < groups.Count; i++)
                    {
                        var transformedLink = TransformIndentedIdLink(groups[i]);
                        childValues.Add(transformedLink);
                    }

                    var linkWithChildren = new Link<TLinkAddress>(link.Id, childValues);
                    list.Add(linkWithChildren);
                }
                else
                {
                    var transformedGroups = new List<LinksGroup<TLinkAddress>>();
                    for (int i = 0; i < groups.Count; i++)
                    {
                        var childGroup = groups[i];
```

```
                        var childLink = childGroup.Link;
                        var childGroups = childGroup.Groups;

                        if (childLink.Id != null &&
                            (childLink.Values == null || childLink.Values.Count == 0) &&
                            childGroups != null && childGroups.Count > 0)
                        {
                            var transformedLink = TransformIndentedIdLink(childGroup);
                            transformedGroups.Add(new LinksGroup<TLinkAddress>(transformedLink));
                        }
                        else
                        {
                            transformedGroups.Add(childGroup);
                        }
                    }

                    var currentDependency = parentDependency.HasValue ? parentDependency.Value.Combine(link) : link;
                    list.Add(currentDependency);

                    for (int i = 0; i < transformedGroups.Count; i++)
                    {
                        CollectLinksWithIndentedIdSyntaxSupport(list, transformedGroups[i], currentDependency);
                    }
                }
            }
            else
            {
                var currentLink = parentDependency.HasValue ? parentDependency.Value.Combine(link) : link;
                list.Add(currentLink);
            }
        }

        private static Link<TLinkAddress> TransformIndentedIdLink<TLinkAddress>(LinksGroup<TLinkAddress> group)
        {
            var link = group.Link;
            var groups = group.Groups;

            if (groups != null && groups.Count > 0 &&
                link.Id != null &&
                (link.Values == null || link.Values.Count == 0))
            {
                var childValues = new List<Link<TLinkAddress>>();
                for (int i = 0; i < groups.Count; i++)
                {
                    var transformedChild = TransformIndentedIdLink(groups[i]);
                    childValues.Add(transformedChild);
                }
                return new Link<TLinkAddress>(link.Id, childValues);
            }
            else if (link.Values != null && link.Values.Count == 1)
            {
                return link.Values[0];
            }
            else
            {
                return link;
            }
        }
    }
}
```

1.4 ./Link.Foundation.Links.Notation/IListExtensions.cs

```
using Platform.Collections;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.CompilerServices;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Provides extension methods for formatting collections of <see cref="Link{TLinkAddress}"/> instances.
    /// </summary>
    public static class IListExtensions
```

```
    {
        /// <summary>
        /// Formats a collection of links as a multi-line string with each link on a separate line.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers.</typeparam>
        /// <param name="links">The collection of links to format.</param>
        /// <returns>A multi-line string representation of the links.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links) => string.Join(Environment.NewLine, links.Select(l => l.ToString()));

        /// <summary>
        /// Formats a collection of links as a multi-line string with optional parentheses trimming for cleaner output.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers.</typeparam>
        /// <param name="links">The collection of links to format.</param>
        /// <param name="lessParentheses">True to remove outer parentheses from each link for cleaner formatting; false to use standard formatting.</param>
        /// <returns>A multi-line string representation of the links.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links, bool lessParentheses)
        {
            if (lessParentheses == false)
            {
                return links.Format();
            }
            else
            {
                return string.Join(Environment.NewLine, links.Select(l => l.ToString().TrimSingle('(').TrimSingle(')')));
            }
        }

        /// <summary>
        /// Formats a collection of links using FormatOptions configuration.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers.</typeparam>
        /// <param name="links">The collection of links to format.</param>
        /// <param name="options">The FormatOptions to use for formatting.</param>
        /// <returns>A multi-line string representation of the links.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links, FormatOptions options)
        {
            if (links == null || links.Count == 0)
            {
                return string.Empty;
            }

            // Apply consecutive link grouping if enabled
            var linksToFormat = options.GroupConsecutive ? GroupConsecutiveLinks(links) : links;

            // Format each link with options
            var formattedLinks = linksToFormat.Select(l => l.FormatWithOptions(options));
            return string.Join(Environment.NewLine, formattedLinks);
        }

        /// <summary>
        /// Group consecutive links with the same ID.
        /// </summary>
        private static List<Link<TLinkAddress>> GroupConsecutiveLinks<TLinkAddress>(IList<Link<TLinkAddress>> links)
        {
            if (links == null || links.Count == 0)
            {
                return new List<Link<TLinkAddress>>();
            }

            var grouped = new List<Link<TLinkAddress>>();
            int i = 0;

            while (i < links.Count)
```

```
            {
                var current = links[i];

                // Look ahead for consecutive links with same ID
                if (current.Id != null && current.Values != null && current.Values.Count > 0)
                {
                    // Collect all values with same ID
                    var sameIdValues = new List<Link<TLinkAddress>>(current.Values);
                    int j = i + 1;

                    while (j < links.Count)
                    {
                        var nextLink = links[j];
                        if (EqualityComparer<TLinkAddress>.Default.Equals(nextLink.Id, current.Id) &&
                            nextLink.Values != null && nextLink.Values.Count > 0)
                        {
                            sameIdValues.AddRange(nextLink.Values);
                            j++;
                        }
                        else
                        {
                            break;
                        }
                    }

                    // If we found consecutive links, create grouped link
                    if (j > i + 1)
                    {
                        var groupedLink = new Link<TLinkAddress>(current.Id, sameIdValues);
                        grouped.Add(groupedLink);
                        i = j;
                        continue;
                    }
                }

                grouped.Add(current);
                i++;
            }

            return grouped;
        }
    }
}
```

1.5 ./Link.Foundation.Links.Notation/Link.cs

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections;
using Platform.Collections.Lists;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Represents a link in the Links Notation with an optional identifier and nested values.
    /// Links can represent simple references, complex nested structures, or relationships between references to links.
    /// </summary>
    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
    public struct Link<TLinkAddress> : IEquatable<Link<TLinkAddress>>
    {
        private static readonly EqualityComparer<TLinkAddress> EqualityComparerInstance = EqualityComparer<TLinkAddress>.Default;

        /// <summary>
        /// Gets the identifier or address of this link. Can be null for anonymous links.
        /// </summary>
        public readonly TLinkAddress? Id;

        /// <summary>
        /// Gets the collection of nested link values. Can be null or empty for simple reference links.
        /// </summary>
        public readonly IList<Link<TLinkAddress>>? Values;

        /// <summary>
        /// Initializes a new link with the specified identifier and values.
```

```
        /// </summary>
        /// <param name="id">The link identifier or address.</param>
        /// <param name="values">The nested link values.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(TLinkAddress? id, IList<Link<TLinkAddress>>? values) => (Id, Values) = (id, values);

        /// <summary>
        /// Initializes a new anonymous link with the specified values.
        /// </summary>
        /// <param name="values">The nested link values.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(IList<Link<TLinkAddress>> values) : this(default!, values) { }

        /// <summary>
        /// Initializes a new anonymous link with the specified array of values.
        /// </summary>
        /// <param name="values">The nested link values as a parameter array.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(params Link<TLinkAddress>[] values) : this(default!, values) { }

        /// <summary>
        /// Initializes a new simple reference link with the specified identifier.
        /// </summary>
        /// <param name="id">The link identifier or address.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(TLinkAddress id) : this(id, default!) { }

        /// <summary>
        /// Returns the string representation of this link in Links Notation format.
        /// </summary>
        /// <returns>A string representation of the link with proper escaping and formatting.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override string ToString() => Id == null ?
         $"({GetValuesString()})" :
            (Values == null || Values.Count == 0) ?
                $"({EscapeReference(Id.ToString())})" :
                $"({EscapeReference(Id.ToString())}: {GetValuesString()})";

        /// <summary>
        /// Gets the string representation of the link's values.
        /// </summary>
        /// <returns>A space-separated string of the values, or empty string if no values exist.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public string GetValuesString()
        {
            if (Values == null || Values.Count == 0)
            {
                return "";
            }
            var sb = new StringBuilder();
            for (int i = 0; i < Values.Count; i++)
            {
                if (i > 0)
                {
                    sb.Append(' ');
                }
                sb.Append(GetValueString(Values[i]));
            }
            return sb.ToString();
        }

        /// <summary>
        /// Simplifies the link structure by unwrapping single-value containers and recursively simplifying nested values.
        /// </summary>
        /// <returns>A simplified version of this link.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link<TLinkAddress> Simplify()
        {
            if (Values == null || Values.Count == 0)
            {
                return this;
            }
            else if (Values.Count == 1)
            {
```

```
                return Values[0];
            }
            else
            {
                var newValues = new Link<TLinkAddress>[Values.Count];
                for (int i = 0; i < Values.Count; i++)
                {
                    newValues[i] = Values[i].Simplify();
                }
                return new Link<TLinkAddress>(Id, newValues);
            }
        }

        /// <summary>
        /// Combines this link with another link to create a new link containing both as values.
        /// </summary>
        /// <param name="other">The other link to combine with.</param>
        /// <returns>A new link containing both links as values.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link<TLinkAddress> Combine(Link<TLinkAddress> other) => new Link<TLinkAddress>(this, other);

        /// <summary>
        /// Gets the string representation of a link value, choosing the appropriate format.
        /// </summary>
        /// <param name="value">The link value to convert to string.</param>
        /// <returns>The string representation of the link value.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string GetValueString(Link<TLinkAddress> value) => value.ToLinkOrIdString();

        /// <summary>
        /// Escapes a reference string for safe use in Links Notation format by adding quotes if necessary.
        /// </summary>
        /// <param name="reference">The reference string to escape.</param>
        /// <returns>The escaped reference string with appropriate quoting.</returns>
        public static string EscapeReference(string? reference)
        {
            if (string.IsNullOrWhiteSpace(reference))
            {
                return "";
            }
            if (
                    reference.Contains(":") ||
                    reference.Contains("(") ||
                    reference.Contains(")") ||
                    reference.Contains(" ") ||
                    reference.Contains("\t") ||
                    reference.Contains("\n") ||
                    reference.Contains("\r") ||
                    reference.Contains("\"")
                )
            {
                return $"'{reference}'";
            }
            else if (reference.Contains("'"))
            {
                return $"\"{reference}\"";
            }
            else
            {
                return reference;
            }
        }

        /// <summary>
        /// Converts the link to its string representation, choosing between simple reference format or full link format.
        /// </summary>
        /// <returns>A string representation optimized for the link's content.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public string ToLinkOrIdString() => (Values == null || Values.Count == 0) ? (Id == null ? "" : EscapeReference(Id?.ToString())) : ToString();

        /// <summary>
        /// Implicitly converts a value to a simple reference link.
        /// </summary>
```

```
        /// <param name="value">The value to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>(TLinkAddress value) => new Link<TLinkAddress>(value);

        /// <summary>
        /// Implicitly converts a tuple of identifier and values to a link.
        /// </summary>
        /// <param name="value">The tuple containing identifier and values.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((TLinkAddress, IList<Link<TLinkAddress>>) value) => new(value.Item1, value.Item2);

        /// <summary>
        /// Implicitly converts a source-target pair to an anonymous link.
        /// </summary>
        /// <param name="value">The tuple containing source and target links.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((Link<TLinkAddress> source, Link<TLinkAddress> target) value) => new(value.source, value.target);

        /// <summary>
        /// Implicitly converts a tuple of identifier, source, and target to a link.
        /// </summary>
        /// <param name="value">The tuple containing id, source and target.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((id<TLinkAddress> id, Link<TLinkAddress> source, Link<TLinkAddress> target) value) => new(value.id.Id, new[] { value.source, value.target });

        /// <summary>
        /// Implicitly converts a source-linker-target triplet to an anonymous link.
        /// </summary>
        /// <param name="value">The tuple containing source, linker, and target links.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((Link<TLinkAddress> source, Link<TLinkAddress> linker, Link<TLinkAddress> target) value) => new(value.source, value.linker, value.target);

        /// <summary>
        /// Implicitly converts a tuple of identifier, source, linker, and target to a link.
        /// </summary>
        /// <param name="value">The tuple containing id, source, linker, and target.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((id<TLinkAddress> id, Link<TLinkAddress> source, Link<TLinkAddress> linker, Link<TLinkAddress> target) value) => new(value.id.Id, new[] { value.source, value.linker, value.target });

        /// <summary>
        /// Determines whether the specified object is equal to this link.
        /// </summary>
        /// <param name="obj">The object to compare with this link.</param>
        /// <returns>True if the specified object is equal to this link; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override bool Equals(object? obj) => obj is Link<TLinkAddress> link && Equals(link);

        /// <summary>
        /// Returns the hash code for this link.
        /// </summary>
        /// <returns>A hash code for this link.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override int GetHashCode() => (Id, (Values ?? Array.Empty<Link<TLinkAddress>>()).GenerateHashCode()).GetHashCode();

        /// <summary>
        /// Indicates whether the current link is equal to another link.
        /// Two anonymous links (both with null Ids) are considered equal if their values are equal.
        /// </summary>
        /// <param name="other">The link to compare with this link.</param>
        /// <returns>True if the current link is equal to the other parameter; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public bool Equals(Link<TLinkAddress> other)
        {
```

```
            // Both have null IDs - compare only values
            if (Id == null && other.Id == null)
            {
                return (Values ?? Array.Empty<Link<TLinkAddress>>()).EqualTo(other.Values ?? Array.Empty<Link<TLinkAddress>>());
            }

            // Only one has null ID - not equal
            if (Id == null || other.Id == null)
            {
                return false;
            }

            // Both have IDs - compare IDs and values
            return EqualityComparerInstance.Equals(Id, other.Id) &&
                   (Values ?? Array.Empty<Link<TLinkAddress>>()).EqualTo(other.Values ?? Array.Empty<Link<TLinkAddress>>());
        }

        /// <summary>
        /// Determines whether two link instances are equal.
        /// </summary>
        /// <param name="left">The first link to compare.</param>
        /// <param name="right">The second link to compare.</param>
        /// <returns>True if the links are equal; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator ==(Link<TLinkAddress> left, Link<TLinkAddress> right) => left.Equals(right);

        /// <summary>
        /// Determines whether two link instances are not equal.
        /// </summary>
        /// <param name="left">The first link to compare.</param>
        /// <param name="right">The second link to compare.</param>
        /// <returns>True if the links are not equal; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator !=(Link<TLinkAddress> left, Link<TLinkAddress> right) => !(left == right);
    }
}
```

1.6 ./Link.Foundation.Links.Notation/LinkFormatExtensions.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Provides extension methods for formatting <see cref="Link{TLinkAddress}"/> instances with FormatOptions.
    /// </summary>
    public static class LinkFormatExtensions
    {
        /// <summary>
        /// Format a link using FormatOptions configuration.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers.</typeparam>
        /// <param name="link">The link to format.</param>
        /// <param name="options">The FormatOptions to use.</param>
        /// <returns>Formatted string representation.</returns>
        public static string FormatWithOptions<TLinkAddress>(this Link<TLinkAddress> link, FormatOptions options)
        {
            // Empty link
            if (link.Id == null && (link.Values == null || link.Values.Count == 0))
            {
                return options.LessParentheses ? "" : "()";
            }

            // Link with only ID, no values
            if (link.Values == null || link.Values.Count == 0)
            {
                var escapedId = Link<TLinkAddress>.EscapeReference(link.Id?.ToString());
                return options.LessParentheses && !NeedsParentheses(link.Id?.ToString()) ?
```

```
                    escapedId :
                    $"({escapedId})";
            }

            // Check if we should use indented format
            bool shouldIndent = false;
            if (options.ShouldIndentByRefCount(link.Values.Count))
            {
                shouldIndent = true;
            }
            else
            {
                // Try inline format first
                var valuesStr = string.Join(" ", link.Values.Select(v => GetValueString(v)));
                string testLine;
                if (link.Id != null)
                {
                    var idStr = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
                    testLine = options.LessParentheses ? $"{idStr}: {valuesStr}" : $"({idStr}: {valuesStr})";
                }
                else
                {
                    testLine = options.LessParentheses ? valuesStr : $"({valuesStr})";
                }

                if (options.ShouldIndentByLength(testLine))
                {
                    shouldIndent = true;
                }
            }

            // Format with indentation if needed
            if (shouldIndent && options.PreferInline == false)
            {
                return FormatIndented(link, options);
            }

            // Standard inline formatting
            var values = string.Join(" ", link.Values.Select(v => GetValueString(v)));

            // Link with values only (null id)
            if (link.Id == null)
            {
                if (options.LessParentheses)
                {
                    var allSimple = link.Values.All(v => v.Values == null || v.Values.Count == 0);
                    if (allSimple)
                    {
                        return string.Join(" ", link.Values.Select(v => Link<TLinkAddress>.EscapeReference(v.Id?.ToString())));
                    }
                    return values;
                }
                return $"({values})";
            }

            // Link with ID and values
            var id = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
            var withColon = $"{id}: {values}";
            return options.LessParentheses && !NeedsParentheses(link.Id?.ToString()) ?
                withColon :
                $"({withColon})";
        }

        /// <summary>
        /// Format a link with indentation.
        /// </summary>
        private static string FormatIndented<TLinkAddress>(Link<TLinkAddress> link, FormatOptions options)
        {
            if (link.Id == null)
            {
                // Values only - format each on separate line
                var lines = link.Values.Select(v => options.IndentString + GetValueString(v));
                return string.Join(Environment.NewLine, lines);
            }

```

```
            // Link with ID - format as id:\n  value1\n  value2
            var idStr = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
            var sb = new StringBuilder();
            sb.Append($"{idStr}:");

            foreach (var v in link.Values)
            {
                sb.Append(Environment.NewLine);
                sb.Append(options.IndentString);
                sb.Append(GetValueString(v));
            }

            return sb.ToString();
        }

        /// <summary>
        /// Get the string representation of a value.
        /// </summary>
        private static string GetValueString<TLinkAddress>(Link<TLinkAddress> value)
        {
            return value.ToLinkOrIdString();
        }

        /// <summary>
        /// Check if a string needs to be wrapped in parentheses.
        /// </summary>
        private static bool NeedsParentheses(string s)
        {
            return s != null && (s.Contains(" ") || s.Contains(":") || s.Contains("(") || s.Contains(")"));
        }
    }
}
```

1.7 ./Link.Foundation.Links.Notation/LinksGroup.cs

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using Platform.Collections;
using Platform.Collections.Lists;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Represents a group of links with hierarchical structure, where each group contains a primary link and optional nested groups.
    /// This structure supports the indentation-based syntax of the Links Notation.
    /// </summary>
    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
    public struct LinksGroup<TLinkAddress> : IEquatable<LinksGroup<TLinkAddress>>
    {
        /// <summary>
        /// Gets or sets the primary link of this group.
        /// </summary>
        public Link<TLinkAddress> Link
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            set;
        }

        /// <summary>
        /// Gets or sets the collection of nested groups within this group. Can be null if no nested groups exist.
        /// </summary>
        public IList<LinksGroup<TLinkAddress>>? Groups
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            set;
        }

        /// <summary>
        /// Initializes a new links group with the specified link and nested groups.
        /// </summary>
        /// <param name="link">The primary link of this group.</param>
```

```
        /// <param name="groups">The nested groups within this group.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksGroup(Link<TLinkAddress> link, IList<LinksGroup<TLinkAddress>>? groups)
        {
            Link = link;
            Groups = groups;
        }

        /// <summary>
        /// Initializes a new links group with the specified link and no nested groups.
        /// </summary>
        /// <param name="link">The primary link of this group.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksGroup(Link<TLinkAddress> link) : this(link, null) { }

        /// <summary>
        /// Implicitly converts a links group to a list of links by flattening the hierarchical structure.
        /// </summary>
        /// <param name="value">The links group to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator List<Link<TLinkAddress>>(LinksGroup<TLinkAddress> value) => value.ToLinksList();

        /// <summary>
        /// Converts this links group to a flat list of links, preserving the hierarchical relationships.
        /// </summary>
        /// <returns>A list of links representing the flattened structure of this group.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public List<Link<TLinkAddress>> ToLinksList()
        {
            var list = new List<Link<TLinkAddress>>();
            AppendToLinksList(list);
            return list;
        }

        /// <summary>
        /// Appends this links group to the specified list as flattened links.
        /// </summary>
        /// <param name="list">The list to append the links to.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public void AppendToLinksList(List<Link<TLinkAddress>> list) => AppendToLinksList(list, Link, this);

        /// <summary>
        /// Recursively appends a links group to a list, combining dependencies with nested groups to create a flattened structure.
        /// </summary>
        /// <param name="list">The list to append the links to.</param>
        /// <param name="dependency">The dependency link to combine with nested groups.</param>
        /// <param name="group">The links group to process.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void AppendToLinksList(List<Link<TLinkAddress>> list, Link<TLinkAddress> dependency, LinksGroup<TLinkAddress> group)
        {
            list.Add(dependency);
            var groups = group.Groups;
            if (groups != null && !groups.IsNullOrEmpty())
            {
                for (int i = 0; i < groups.Count; i++)
                {
                    var innerGroup = groups[i];
                    AppendToLinksList(list, dependency.Combine(innerGroup.Link), innerGroup);
                }
            }
        }

        /// <summary>
        /// Determines whether the specified object is equal to this links group.
        /// </summary>
        /// <param name="obj">The object to compare with this links group.</param>
        /// <returns>True if the specified object is equal to this links group; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override bool Equals(object? obj) => obj is LinksGroup<TLinkAddress> linksGroup ? Equals(linksGroup) : false;
```

```

        /// <summary>
        /// Returns the hash code for this links group.
        /// </summary>
        /// <returns>A hash code for this links group.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override int GetHashCode() => (Link, (Groups ??
↪   Array.Empty<LinksGroup<TLinkAddress>>()).GenerateHashCode()).GetHashCode();

        /// <summary>
        /// Indicates whether the current links group is equal to another links group.
        /// </summary>
        /// <param name="other">The links group to compare with this links group.</param>
        /// <returns>True if the current links group is equal to the other parameter; otherwise,
↪   false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public bool Equals(LinksGroup<TLinkAddress> other) => Link.Equals(other.Link) && (Groups
↪   ?? Array.Empty<LinksGroup<TLinkAddress>>()).EqualTo(other.Groups ??
↪   Array.Empty<LinksGroup<TLinkAddress>>());

        /// <summary>
        /// Determines whether two links group instances are equal.
        /// </summary>
        /// <param name="left">The first links group to compare.</param>
        /// <param name="right">The second links group to compare.</param>
        /// <returns>True if the links groups are equal; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator ==(LinksGroup<TLinkAddress> left, LinksGroup<TLinkAddress>
↪   right) => left.Equals(right);

        /// <summary>
        /// Determines whether two links group instances are not equal.
        /// </summary>
        /// <param name="left">The first links group to compare.</param>
        /// <param name="right">The second links group to compare.</param>
        /// <returns>True if the links groups are not equal; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator !=(LinksGroup<TLinkAddress> left, LinksGroup<TLinkAddress>
↪   right) => !(left == right);

        /// <summary>
        /// Returns a string representation of this links group.
        /// </summary>
        /// <returns>A string representation of this links group.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override string ToString() => ToLinksList().Format();
    }
}
```

1.8 ./Link.Foundation.Links.Notation/ .cs

```
using System.Runtime.CompilerServices;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// A utility struct that provides convenient implicit conversions for creating links from
↪   various tuple combinations.
    /// The underscore name follows functional programming conventions for placeholder/utility
↪   types.
    /// </summary>
    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
↪   any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
    public struct  <TLinkAddress>
    {
        /// <summary>
        /// Gets the underlying link represented by this utility struct.
        /// </summary>
        public readonly Link<TLinkAddress> Link;

        /// <summary>
        /// Initializes a new instance with the specified link.
        /// </summary>
        /// <param name="id">The link to wrap.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public  (Link<TLinkAddress> id) => Link = id;

        /// <summary>
        /// Implicitly converts a link to this utility struct.
```

```
        /// </summary>
        /// <param name="value">The link to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>(Link<TLinkAddress> value) => new(value);

        /// <summary>
        /// Implicitly converts a link address to this utility struct.
        /// </summary>
        /// <param name="id">The link address to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>(TLinkAddress id) => new(id);

        /// <summary>
        /// Implicitly converts a tuple of address and link to this utility struct.
        /// </summary>
        /// <param name="value">The tuple containing address and link.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>((TLinkAddress, Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2 });

        /// <summary>
        /// Implicitly converts a tuple of address and two links to this utility struct.
        /// </summary>
        /// <param name="value">The tuple containing address and two links.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>((TLinkAddress, Link<TLinkAddress>, Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2, value.Item3 });

        /// <summary>
        /// Implicitly converts a tuple of address and three links to this utility struct.
        /// </summary>
        /// <param name="value">The tuple containing address and three links.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>((TLinkAddress, Link<TLinkAddress>, Link<TLinkAddress>, Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2, value.Item3, value.Item4 });

        /// <summary>
        /// Implicitly converts this utility struct back to a link.
        /// </summary>
        /// <param name="value">The utility struct to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>( <TLinkAddress> value) => value.Link;
    }
}
```

1.9 ./Link.Foundation.Links.Notation/id.cs

```
using System.Runtime.CompilerServices;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// A readonly struct that explicitly represents a link identifier/address.
    /// This type is used to distinguish between regular values and explicit identifiers in link construction.
    /// </summary>
    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
    public readonly struct id<TLinkAddress>
    {
        /// <summary>
        /// Gets the identifier value.
        /// </summary>
        public readonly TLinkAddress Id;

        /// <summary>
        /// Initializes a new identifier with the specified value.
        /// </summary>
        /// <param name="id">The identifier value.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public id(TLinkAddress id) => Id = id;

        /// <summary>
        /// Explicitly converts a value to an identifier struct.
        /// </summary>
        /// <param name="id">The value to convert to an identifier.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
```

```
        public static explicit operator id<TLinkAddress>(TLinkAddress id) => new(id);
    }
}
```

1.10 ./Link.Foundation.Links.Notation.Tests/ApiTests.cs

```
using System;
using System.Collections.Generic;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class ApiTests
    {
        [Fact]
        public static void IsRefTest()
        {
            // C# doesn't have separate Ref/Link types, but we can test simple link behavior
            var simpleLink = new Link<string>("some value", null);
            Assert.Equal("some value", simpleLink.Id);
            Assert.Null(simpleLink.Values);
        }

        [Fact]
        public static void IsLinkTest()
        {
            // Test link with values
            var values = new List<Link<string>> { new Link<string>("child", null) };
            var link = new Link<string>("id", values);
            Assert.Equal("id", link.Id);
            Assert.Single(link.Values);
            Assert.Equal("child", link.Values?[0].Id);
        }

        [Fact]
        public static void IsRefEquivalentTest()
        {
            // C# doesn't have separate Ref/Link types, but we can test simple link behavior
            var simpleLink = new Link<string>("some value", null);
            Assert.Equal("some value", simpleLink.Id);
            Assert.Null(simpleLink.Values);
        }

        [Fact]
        public static void IsLinkEquivalentTest()
        {
            // Test link with values
            var values = new List<Link<string>> { new Link<string>("child", null) };
            var link = new Link<string>("id", values);
            Assert.Equal("id", link.Id);
            Assert.Single(link.Values);
            Assert.Equal("child", link.Values?[0].Id);
        }

        [Fact]
        public static void EmptyLinkTest()
        {
            var link = new Link<string>(null, new List<Link<string>>());
            var output = link.ToString();
            Assert.Equal("()", output);
        }

        [Fact]
        public static void SimpleLinkTest()
        {
            var input = "(1: 1 1)";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.Contains("1:", output);
            Assert.Contains("1", output);
        }

        [Fact]
        public static void LinkWithSourceTargetTest()
        {
            var input = "(index: source target)";
            var parser = new Parser();
```

```
            var parsed = parser.Parse(input);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.Equal(input, output);
        }

        [Fact]
        public static void LinkWithSourceTypeTargetTest()
        {
            var input = "(index: source type target)";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.Equal(input, output);
        }

        [Fact]
        public static void SingleLineFormatTest()
        {
            var input = "id: value1 value2";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // The parser should handle single-line format
            var output = parsed.Format();
            Assert.Contains("id", output);
            Assert.Contains("value1", output);
            Assert.Contains("value2", output);
        }

        [Fact]
        public static void QuotedReferencesTest()
        {
            var input = @"(""quoted id"": ""value with spaces"")";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            var output = parsed.Format();
            Assert.Contains("quoted id", output);
            Assert.Contains("value with spaces", output);
        }

        [Fact]
        public static void QuotedReferencesParsingTest()
        {
            // Test that quoted references are parsed correctly
            var input = @"(""quoted id"": ""value with spaces"")";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Verify parsing worked correctly
            var output = parsed.Format();
            Assert.Contains("quoted id", output);
            Assert.Contains("value with spaces", output);
        }

        [Fact]
        public static void IndentedIdSyntaxParsingTest()
        {
            // Test that indented ID syntax is parsed correctly
            var indented = "id:\n  value1\n  value2";
            var inline = "(id: value1 value2)";

            var parser = new Parser();
            var indentedParsed = parser.Parse(indented);
            var inlineParsed = parser.Parse(inline);

            // Both should produce equivalent structures
            var indentedOutput = indentedParsed.Format();
            var inlineOutput = inlineParsed.Format();
            Assert.Equal(indentedOutput, inlineOutput);
            Assert.Equal("(id: value1 value2)", indentedOutput);
        }

        [Fact]
        public static void IndentedIdSyntaxRoundtripTest()
```

```
        {
            var input = "id:\n  value1\n  value2";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate that we can format with indented syntax using FormatOptions
            var options = new FormatOptions
            {
                MaxInlineRefs = 1,  // Force indentation with more than 1 ref
                PreferInline = false
            };
            var output = parsed.Format(options);
            Assert.Equal(input, output);
        }

        [Fact]
        public static void MultipleIndentedIdSyntaxParsingTest()
        {
            // Test that multiple indented ID links are parsed correctly
            var indented = "id1:\n  a\n  b\nid2:\n  c\n  d";
            var inline = "(id1: a b)\n(id2: c d)";

            var parser = new Parser();
            var indentedParsed = parser.Parse(indented);
            var inlineParsed = parser.Parse(inline);

            // Both should produce equivalent structures
            var indentedOutput = indentedParsed.Format();
            var inlineOutput = inlineParsed.Format();
            Assert.Equal(indentedOutput, inlineOutput);
            Assert.Equal("(id1: a b)\n(id2: c d)", indentedOutput);
        }

        [Fact]
        public static void MultipleIndentedIdSyntaxRoundtripTest()
        {
            var input = "id1:\n  a\n  b\nid2:\n  c\n  d";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate that we can format with indented syntax using FormatOptions
            var options = new FormatOptions
            {
                MaxInlineRefs = 1,  // Force indentation with more than 1 ref
                PreferInline = false
            };
            var output = parsed.Format(options);
            Assert.Equal(input, output);
        }
    }
}
```

1.11 ./Link.Foundation.Links.Notation.Tests/EdgeCaseParserTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class EdgeCaseParserTests
    {
        [Fact]
        public static void EmptyLinkTest()
        {
            var source = @":";
            var parser = new Parser();
            // Standalone ':' is now forbidden and should throw an exception
            Assert.Throws<FormatException>(() => parser.Parse(source));
        }

        [Fact]
        public static void EmptyLinkWithParenthesesTest()
        {
            var source = @"()";
            var target = @"()";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }
```

```

        [Fact]
        public static void EmptyLinkWithEmptySelfReferenceTest()
        {
            var source = @"(:)";
            var parser = new Parser();
            // '(:)' is now forbidden and should throw an exception
            Assert.Throws<FormatException>(() => parser.Parse(source));
        }

        [Fact]
        public static void AllFeaturesTest()
        {
            // Test single-line link with id
            var input = "id: value1 value2";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test multi-line link with id
            input = "(id: value1 value2)";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test link without id (single-line) - now forbidden
            input = ": value1 value2";
            Assert.Throws<FormatException>(() => new Parser().Parse(input));

            // Test link without id (multi-line) - now forbidden
            input = "(: value1 value2)";
            Assert.Throws<FormatException>(() => new Parser().Parse(input));

            // Test singlet link
            input = "(singlet)";
            result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("singlet", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);

            // Test value link
            input = "(value1 value2 value3)";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test quoted references
            input = @""(""id with spaces"": ""value with spaces"")";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test single-quoted references
            input = "('id': 'value')";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test nested links
            input = "(outer: (inner: value))";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void TestSingletLinks()
        {
            // Test singlet (1)
            var input = "(1)";
            var result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("1", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);

            // Test (1 2)
            input = "(1 2)";
            result = new Parser().Parse(input);
            Assert.Single(result);
```

```
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Equal(2, result[0].Values?.Count);
            Assert.Equal("1", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);
            Assert.Equal("2", result[0].Values?[1].Id);
            Assert.Null(result[0].Values?[1].Values);

            // Test (1 2 3)
            input = "(1 2 3)";
            result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Equal(3, result[0].Values?.Count);
            Assert.Equal("1", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);
            Assert.Equal("2", result[0].Values?[1].Id);
            Assert.Null(result[0].Values?[1].Values);
            Assert.Equal("3", result[0].Values?[2].Id);
            Assert.Null(result[0].Values?[2].Values);

            // Test (1 2 3 4)
            input = "(1 2 3 4)";
            result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Equal(4, result[0].Values?.Count);
            Assert.Equal("1", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);
            Assert.Equal("2", result[0].Values?[1].Id);
            Assert.Null(result[0].Values?[1].Values);
            Assert.Equal("3", result[0].Values?[2].Id);
            Assert.Null(result[0].Values?[2].Values);
            Assert.Equal("4", result[0].Values?[3].Id);
            Assert.Null(result[0].Values?[3].Values);
        }

        [Fact]
        public static void EmptyDocumentTest()
        {
            var input = "";
            // Empty document should return empty list
            var result = new Parser().Parse(input);
            Assert.Empty(result);
        }

        [Fact]
        public static void WhitespaceOnlyTest()
        {
            var input = "   \n   \n   ";
            // Whitespace-only document should return empty list (similar to empty document)
            var result = new Parser().Parse(input);
            Assert.Empty(result);
        }

        [Fact]
        public static void EmptyLinksTest()
        {
            var input = "()";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // '(:)' is now forbidden
            input = "(:)";
            Assert.Throws<FormatException>(() => new Parser().Parse(input));

            input = "(id:)";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void InvalidInputTest()
        {
            var input = "(invalid";
            // Unclosed parentheses should throw an exception
```

```
            Assert.Throws<FormatException>(() => new Parser().Parse(input));
        }
    }
}
```

1.12 ./Link.Foundation.Links.Notation.Tests/FormatConfigTests.cs

```
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public class FormatConfigTests
    {
        [Fact]
        public void FormatConfigBasicTest()
        {
            var config = new FormatConfig();
            Assert.False(config.LessParentheses);
            Assert.Equal(80, config.MaxLineLength);
            Assert.False(config.IndentLongLines);
        }

        [Fact]
        public void FormatWithLineLengthLimitTest()
        {
            // Create a config with line length limit
            // The line would be 32+ chars, so use threshold of 30
            var config = new FormatConfig
            {
                IndentLongLines = true,
                MaxLineLength = 30,
                PreferInline = false
            };

            // Verify config is set correctly
            Assert.True(config.IndentLongLines);
            Assert.Equal(30, config.MaxLineLength);
            // Note: Full formatting integration would test actual output here
        }

        [Fact]
        public void FormatWithMaxInlineRefsTest()
        {
            // Create a config with MaxInlineRefs=3 (should trigger indentation with 4 refs)
            var config = new FormatConfig
            {
                MaxInlineRefs = 3,
                PreferInline = false
            };

            // Verify config is set correctly
            Assert.Equal(3, config.MaxInlineRefs);
            Assert.True(config.ShouldIndentByRefCount(4));
            // Note: Full formatting integration would test actual output here
        }

        [Fact]
        public void FormatWithConsecutiveGroupingTest()
        {
            // Create a config with consecutive grouping enabled
            var config = new FormatConfig
            {
                GroupConsecutive = true
            };

            // Verify config is set correctly
            Assert.True(config.GroupConsecutive);
            // Note: Full formatting integration would test grouping behavior here
        }

        [Fact]
        public void FormatConfigCustomIndentTest()
        {
            var config = new FormatConfig
            {
                MaxInlineRefs = 3,
                PreferInline = false,
                IndentString = "    " // 4 spaces
            };

            Assert.Equal("    ", config.IndentString);
```

```
        }

        [Fact]
        public void FormatConfigLessParenthesesTest()
        {
            var config = new FormatConfig
            {
                LessParentheses = true
            };

            Assert.True(config.LessParentheses);
        }

        [Fact]
        public void RoundtripWithLineLengthFormattingTest()
        {
            // This test would require full format integration
            // For now, just verify config creation
            var config = new FormatConfig
            {
                MaxInlineRefs = 2,
                PreferInline = false
            };

            Assert.Equal(2, config.MaxInlineRefs);
        }

        [Fact]
        public void ShouldIndentByLengthTest()
        {
            var config = new FormatConfig
            {
                IndentLongLines = true,
                MaxLineLength = 80
            };

            Assert.False(config.ShouldIndentByLength("short"));
            Assert.True(config.ShouldIndentByLength(new string('a', 100)));
        }

        [Fact]
        public void ShouldIndentByRefCountTest()
        {
            var config = new FormatConfig
            {
                MaxInlineRefs = 3
            };

            Assert.False(config.ShouldIndentByRefCount(2));
            Assert.False(config.ShouldIndentByRefCount(3));
            Assert.True(config.ShouldIndentByRefCount(4));
        }
    }
}
```

1.13 ./Link.Foundation.Links.Notation.Tests/IndentationConsistencyTests.cs

```
using Xunit;
using System.Collections.Generic;

namespace Link.Foundation.Links.Notation.Tests
{
    public class IndentationConsistencyTests
    {
        [Fact]
        public void LeadingSpacesVsNoLeadingSpaces()
        {
            // Example with 2 leading spaces (from issue #135)
            var withLeading = @"  TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
  TELEGRAM ALLOWED CHATS:
    -1002975819706
    -1002861722681
  TELEGRAM HIVE OVERRIDES:
    --all-issues
    --once
  TELEGRAM BOT VERBOSE: true";

            // Example without leading spaces (from issue #135)
            var withoutLeading = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
TELEGRAM ALLOWED CHATS:
  -1002975819706
  -1002861722681
```

```
TELEGRAM HIVE OVERRIDES:
  --all-issues
  --once
TELEGRAM BOT VERBOSE: true";

            var resultWith = new Parser().Parse(withLeading);
            var resultWithout = new Parser().Parse(withoutLeading);

            // Compare the entire formatted output (complete round trip test)
            Assert.Equal(resultWithout.Format(), resultWith.Format());
        }

        [Fact]
        public void TwoSpacesVsFourSpacesIndentation()
        {
            // Example with 2 spaces per level
            var twoSpaces = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
TELEGRAM ALLOWED CHATS:
  -1002975819706
  -1002861722681
TELEGRAM HIVE OVERRIDES:
  --all-issues
  --once
  --auto-fork
  --skip-issues-with-prs
  --attach-logs
  --verbose
  --no-tool-check
TELEGRAM SOLVE OVERRIDES:
  --auto-fork
  --auto-continue
  --attach-logs
  --verbose
  --no-tool-check
TELEGRAM BOT VERBOSE: true";

            // Example with 4 spaces per level
            var fourSpaces = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
TELEGRAM ALLOWED CHATS:
    -1002975819706
    -1002861722681
TELEGRAM HIVE OVERRIDES:
    --all-issues
    --once
    --auto-fork
    --skip-issues-with-prs
    --attach-logs
    --verbose
    --no-tool-check
TELEGRAM SOLVE OVERRIDES:
    --auto-fork
    --auto-continue
    --attach-logs
    --verbose
    --no-tool-check
TELEGRAM BOT VERBOSE: true";

            var resultTwo = new Parser().Parse(twoSpaces);
            var resultFour = new Parser().Parse(fourSpaces);

            // Compare the entire formatted output (complete round trip test)
            Assert.Equal(resultFour.Format(), resultTwo.Format());
        }

        [Fact]
        public void SimpleTwoVsFourSpacesIndentation()
        {
            // Simple example with 2 spaces
            var twoSpaces = @"parent:
  child1
  child2";

            // Simple example with 4 spaces
            var fourSpaces = @"parent:
    child1
    child2";

            var resultTwo = new Parser().Parse(twoSpaces);
            var resultFour = new Parser().Parse(fourSpaces);

            // Compare the entire formatted output (complete round trip test)
            Assert.Equal(resultFour.Format(), resultTwo.Format());
        }
```

```

        [Fact]
        public void ThreeLevelNestingWithDifferentIndentation()
        {
            // Three levels with 2 spaces
            var twoSpaces = @"level1:
  level2:
    level3a
    level3b
  level2b";

            // Three levels with 4 spaces
            var fourSpaces = @"level1:
    level2:
        level3a
        level3b
    level2b";

            var resultTwo = new Parser().Parse(twoSpaces);
            var resultFour = new Parser().Parse(fourSpaces);

            // Compare the entire formatted output (complete round trip test)
            Assert.Equal(resultFour.Format(), resultTwo.Format());
        }
    }
}
```

1.14 ./Link.Foundation.Links.Notation.Tests/IndentedIdSyntaxTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class IndentedIdSyntaxTests
    {
        [Fact]
        public static void BasicIndentedIdSyntaxTest()
        {
            var indentedSyntax = @"3:
  papa
  loves
  mama";
            var inlineSyntax = "(3: papa loves mama)";

            var parser = new Parser();
            var indentedResult = parser.Parse(indentedSyntax);
            var inlineResult = parser.Parse(inlineSyntax);

            var indentedFormatted = indentedResult.Format();
            var inlineFormatted = inlineResult.Format();

            Assert.Equal(inlineFormatted, indentedFormatted);
            Assert.Equal("(3: papa loves mama)", indentedFormatted);
        }

        [Fact]
        public static void IndentedIdSyntaxWithSingleValueTest()
        {
            var input = @"greeting:
  hello";

            var parser = new Parser();
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal("(greeting: hello)", formatted);
            Assert.Single(result);
            Assert.Equal("greeting", result[0].Id);
            Assert.Single(result[0].Values);
            Assert.Equal("hello", result[0].Values[0].Id);
        }

        [Fact]
        public static void IndentedIdSyntaxWithMultipleValuesTest()
        {
            var input = @"action:
  run
  fast
  now";

            var parser = new Parser();
```

```
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal("(action: run fast now)", formatted);
            Assert.Single(result);
            Assert.Equal("action", result[0].Id);
            Assert.Equal(3, result[0].Values.Count);
        }

        [Fact]
        public static void IndentedIdSyntaxWithNumericIdTest()
        {
            var input = @"42:
  answer
  to
  everything";

            var parser = new Parser();
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal("(42: answer to everything)", formatted);
        }

        [Fact]
        public static void IndentedIdSyntaxWithQuotedIdTest()
        {
            var input = @"""complex id"":
  value1
  value2";

            var parser = new Parser();
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal("('complex id': value1 value2)", formatted);
        }

        [Fact]
        public static void MultipleIndentedIdLinksTest()
        {
            var input = @"first:
  a
  b
second:
  c
  d";

            var parser = new Parser();
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal(2, result.Count);
            Assert.Contains("(first: a b)", formatted);
            Assert.Contains("(second: c d)", formatted);
        }

        [Fact]
        public static void MixedIndentedAndRegularSyntaxTest()
        {
            var input = @"first:
  a
  b
(second: c d)
third value";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Equal(3, result.Count);

            var formatted = result.Format();
            Assert.Contains("(first: a b)", formatted);
            Assert.Contains("(second: c d)", formatted);
            Assert.Contains("third value", formatted);
        }

        [Fact]
        public static void UnsupportedColonOnlySyntaxShouldFailTest()
        {
```

```
            var input = @":
  papa
  loves
  mama";

            var parser = new Parser();
            Assert.Throws<FormatException>(() => parser.Parse(input));
        }

        [Fact]
        public static void EmptyIndentedIdShouldWorkTest()
        {
            var input = "empty:";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Equal("empty", result[0].Id);
            Assert.True(result[0].Values == null || result[0].Values.Count == 0);

            var formatted = result.Format();
            Assert.Equal("(empty)", formatted);
        }

        [Fact]
        public static void EquivalenceTestComprehensiveTest()
        {
            var testCases = new[]
            {
                new { Indented = "test:\n  one", Inline = "(test: one)" },
                new { Indented = "x:\n  a\n  b\n  c", Inline = "(x: a b c)" },
                new { Indented = "\"quoted\":\n  value", Inline = "(\"quoted\": value)" }
            };

            var parser = new Parser();

            foreach (var testCase in testCases)
            {
                var indentedResult = parser.Parse(testCase.Indented);
                var inlineResult = parser.Parse(testCase.Inline);

                var indentedFormatted = indentedResult.Format();
                var inlineFormatted = inlineResult.Format();

                Assert.Equal(inlineFormatted, indentedFormatted);
            }
        }

        [Fact]
        public static void IndentedIdWithDeeperNestingTest()
        {
            var input = @"root:
  child1
  child2
    grandchild";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);

            var rootLink = result[0];
            Assert.Equal("root", rootLink.Id);
            Assert.Equal(2, rootLink.Values.Count);
        }
    }
}
```

1.15 ./Link.Foundation.Links.Notation.Tests/LinkTests.cs

```
using System;
using Xunit;
using System.Collections.Generic;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class LinkTests
    {
        [Fact]
        public static void LinkConstructorWithIdOnlyTest()
```

```
        {
            var link = new Link<string>("test", null);
            Assert.Equal("test", link.Id);
            Assert.Null(link.Values);
        }

        [Fact]
        public static void LinkConstructorWithIdAndValuesTest()
        {
            var values = new List<Link<string>> { new Link<string>("value1"), new Link<string>("value2") };
            var link = new Link<string>("parent", values);
            Assert.Equal("parent", link.Id);
            Assert.Equal(2, link.Values?.Count);
        }

        [Fact]
        public static void LinkToStringWithIdOnlyTest()
        {
            var link = new Link<string>("test");
            Assert.Equal("(test)", link.ToString());
        }

        [Fact]
        public static void LinkToStringWithValuesOnlyTest()
        {
            var values = new List<Link<string>> { new Link<string>("value1"), new Link<string>("value2") };
            var link = new Link<string>(null, values);
            Assert.Equal("(value1 value2)", link.ToString());
        }

        [Fact]
        public static void LinkToStringWithIdAndValuesTest()
        {
            var values = new List<Link<string>> { new Link<string>("child1"), new Link<string>("child2") };
            var link = new Link<string>("parent", values);
            Assert.Equal("(parent: child1 child2)", link.ToString());
        }

        [Fact]
        public static void LinkEqualsTest()
        {
            var link1 = new Link<string>("test");
            var link2 = new Link<string>("test");
            var link3 = new Link<string>("other");

            Assert.Equal(link1, link2);
            Assert.NotEqual(link1, link3);
        }

        [Fact]
        public static void LinkCombineTest()
        {
            var link1 = new Link<string>("first");
            var link2 = new Link<string>("second");
            var combined = link1.Combine(link2);

            Assert.Null(combined.Id);
            Assert.Equal(2, combined.Values?.Count);
            Assert.Equal("first", combined.Values?[0].Id);
            Assert.Equal("second", combined.Values?[1].Id);
        }

        [Fact]
        public static void LinkEscapeReferenceSimpleTest()
        {
            Assert.Equal("test", Link<string>.EscapeReference("test"));
            Assert.Equal("test123", Link<string>.EscapeReference("test123"));
        }

        [Fact]
        public static void LinkEscapeReferenceWithSpecialCharactersTest()
        {
            Assert.Equal("'has space'", Link<string>.EscapeReference("has space"));
            Assert.Equal("'has:colon'", Link<string>.EscapeReference("has:colon"));
            Assert.Equal("'has(paren)'", Link<string>.EscapeReference("has(paren)"));
```

```
            Assert.Equal("'has)paren'", Link<string>.EscapeReference("has)paren"));
            Assert.Equal("\"has'quote\"", Link<string>.EscapeReference("has'quote"));
            Assert.Equal("'has\"quote'", Link<string>.EscapeReference("has\"quote"));
        }

        [Fact]
        public static void LinkSimplifyTest()
        {
            // Test simplify with empty values
            var link1 = new Link<string>("test", null);
            var simplified1 = link1.Simplify();
            Assert.Equal(link1, simplified1);

            // Test simplify with single value
            var link2 = new Link<string>(null, new List<Link<string>> { new Link<string>("single", null) });
            var simplified2 = link2.Simplify();
            Assert.Equal("single", simplified2.Id);
            Assert.Null(simplified2.Values);
        }
    }
}
```

1.16 ./Link.Foundation.Links.Notation.Tests/LinksGroupTests.cs

```
using System;
using System.Collections.Generic;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class LinksGroupTests
    {
        [Fact]
        public static void LinksGroupConstructorTest()
        {
            var element = new Link<string>("root", null);
            var groups = new List<LinksGroup<string>>
            {
                new LinksGroup<string>(new Link<string>("child1", null)),
                new LinksGroup<string>(new Link<string>("child2", null))
            };
            var group = new LinksGroup<string>(element, groups);

            Assert.Equal(element, group.Link);
            Assert.Equal(groups, group.Groups);
        }

        [Fact]
        public static void LinksGroupToListFlattensStructureTest()
        {
            var root = new Link<string>("root", null);
            var child1 = new Link<string>("child1", null);
            var child2 = new Link<string>("child2", null);
            var grandchild = new Link<string>("grandchild", null);

            var childGroup = new LinksGroup<string>(child2, new List<LinksGroup<string>>
            {
                new LinksGroup<string>(grandchild)
            });

            var group = new LinksGroup<string>(root, new List<LinksGroup<string>>
            {
                new LinksGroup<string>(child1),
                childGroup
            });

            var list = group.ToLinksList();
            // The C# implementation creates a hierarchical structure
            // root, root.Combine(child1), root.Combine(child2), root.Combine(child2).Combine(grandchild)
            Assert.Equal(4, list.Count);
            Assert.Equal(root, list[0]);
        }

        [Fact]
        public static void LinksGroupAppendToLinksListTest()
        {
            var element = new Link<string>("root", null);
            var children = new List<LinksGroup<string>>
```

```
            {
                new LinksGroup<string>(new Link<string>("child1", null)),
                new LinksGroup<string>(new Link<string>("child2", null))
            };
            var group = new LinksGroup<string>(element, children);

            var list = new List<Link<string>>();
            group.AppendToLinksList(list);

            Assert.Equal(3, list.Count);
            Assert.Equal(element, list[0]);
        }

        [Fact]
        public static void LinksGroupToStringTest()
        {
            var element = new Link<string>("root", null);
            var children = new List<LinksGroup<string>>
            {
                new LinksGroup<string>(new Link<string>("child1", null)),
                new LinksGroup<string>(new Link<string>("child2", null))
            };
            var group = new LinksGroup<string>(element, children);

            var str = group.ToString();
            Assert.Contains("root", str);
            Assert.Contains("child1", str);
            Assert.Contains("child2", str);
        }

        [Fact]
        public static void LinksGroupConstructorEquivalentTest()
        {
            // Test creating a LinksGroup structure in an equivalent way
            var root = new Link<string>("root", null);
            var children = new List<Link<string>>
            {
                new Link<string>("child1", null),
                new Link<string>("child2", null)
            };

            // Create a group with an id
            var groupElement = new Link<string>("group", null);
            var childGroups = new List<LinksGroup<string>>
            {
                new LinksGroup<string>(root),
                new LinksGroup<string>(children[0]),
                new LinksGroup<string>(children[1])
            };
            var group = new LinksGroup<string>(groupElement, childGroups);

            Assert.Equal("group", group.Link.Id);
            Assert.Equal(3, group.Groups.Count);
            Assert.Equal(root, group.Groups[0].Link);
        }
    }
}
```

1.17 ./Link.Foundation.Links.Notation.Tests/MixedIndentationModesTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class MixedIndentationModesTests
    {
        [Fact]
        public static void HeroExampleMixedModesTest()
        {
            var input = @"empInfo
  employees:
    (
      name (James Kirk)
      age 40
    )
    (
      name (Jean-Luc Picard)
      age 45
    )
```

```
    (
      name (Wesley Crusher)
      age 27
    )";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("empInfo", formatted);
            Assert.Contains("employees", formatted);
            Assert.Contains("James Kirk", formatted);
            Assert.Contains("Jean-Luc Picard", formatted);
            Assert.Contains("Wesley Crusher", formatted);
        }

        [Fact]
        public static void HeroExampleAlternativeFormatTest()
        {
            var input = @"empInfo
  (
    employees:
      (
        name (James Kirk)
        age 40
      )
      (
        name (Jean-Luc Picard)
        age 45
      )
      (
        name (Wesley Crusher)
        age 27
      )
  )";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("empInfo", formatted);
            Assert.Contains("employees:", formatted);
            Assert.Contains("James Kirk", formatted);
            Assert.Contains("Jean-Luc Picard", formatted);
            Assert.Contains("Wesley Crusher", formatted);
        }

        [Fact]
        public static void HeroExampleEquivalenceTest()
        {
            var version1 = @"empInfo
  employees:
    (
      name (James Kirk)
      age 40
    )
    (
      name (Jean-Luc Picard)
      age 45
    )
    (
      name (Wesley Crusher)
      age 27
    )";

            var version2 = @"empInfo
  (
    employees:
      (
        name (James Kirk)
        age 40
      )
      (
        name (Jean-Luc Picard)
        age 45
      )
      (
```

```
        name (Wesley Crusher)
        age 27
      )
  )";

            var parser = new Parser();
            var result1 = parser.Parse(version1);
            var result2 = parser.Parse(version2);

            Assert.NotEmpty(result1);
            Assert.NotEmpty(result2);

            var formatted1 = result1.Format();
            var formatted2 = result2.Format();

            Assert.Equal(formatted1, formatted2);
        }

        [Fact]
        public static void SetContextWithoutColonTest()
        {
            var input = @"empInfo
  employees";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("empInfo", formatted);
            Assert.Contains("employees", formatted);
        }

        [Fact]
        public static void SequenceContextWithColonTest()
        {
            var input = @"employees:
  James Kirk
  Jean-Luc Picard
  Wesley Crusher";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);
            var formatted = result.Format();
            Assert.Contains("employees:", formatted);
            Assert.Contains("James Kirk", formatted);
            Assert.Contains("Jean-Luc Picard", formatted);
            Assert.Contains("Wesley Crusher", formatted);
        }

        [Fact]
        public static void SequenceContextWithComplexValuesTest()
        {
            var input = @"employees:
  (
    name (James Kirk)
    age 40
  )
  (
    name (Jean-Luc Picard)
    age 45
  )";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);
            var formatted = result.Format();
            Assert.Contains("employees:", formatted);
            Assert.Contains("James Kirk", formatted);
            Assert.Contains("Jean-Luc Picard", formatted);
        }

        [Fact]
        public static void NestedSetAndSequenceContextsTest()
        {
```

```
            var input = @"company
  departments:
    engineering
    sales
  employees:
    (name John)
    (name Jane)";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("company", formatted);
            Assert.Contains("departments:", formatted);
            Assert.Contains("employees:", formatted);
        }

        [Fact]
        public static void DeeplyNestedMixedModesTest()
        {
            var input = @"root
  level1
    level2:
      value1
      value2
    level2b
      level3";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("root", formatted);
            Assert.Contains("level2:", formatted);
        }
    }
}
```

1.18 ./Link.Foundation.Links.Notation.Tests/MultiQuoteParserTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class MultiQuoteParserTests
    {
        // Helper to extract single reference ID
        private static string? GetSingleRefId(System.Collections.Generic.IList<Link<string>>
↪   result)
        {
            if (result.Count == 1 && result[0].Id == null && result[0].Values?.Count == 1)
            {
                return result[0].Values[0].Id;
            }
            return result.Count == 1 ? result[0].Id : null;
        }

        // ============================================================================
        // Backtick Quote Tests (Single Backtick)
        // ============================================================================

        [Fact]
        public static void TestBacktickQuotedReference()
        {
            var parser = new Parser();
            var result = parser.Parse("`backtick quoted`");
            Assert.Equal("backtick quoted", GetSingleRefId(result));
        }

        [Fact]
        public static void TestBacktickQuotedWithSpaces()
        {
            var parser = new Parser();
            var result = parser.Parse("`text with spaces`");
            Assert.Equal("text with spaces", GetSingleRefId(result));
        }

        [Fact]
```

```
        public static void TestBacktickQuotedMultiline()
        {
            var parser = new Parser();
            var result = parser.Parse("(`line1\nline2`)");
            Assert.Single(result);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("line1\nline2", result[0].Values![0].Id);
        }

        [Fact]
        public static void TestBacktickQuotedWithEscapedBacktick()
        {
            var parser = new Parser();
            var result = parser.Parse("`text with `` escaped backtick`");
            Assert.Equal("text with ` escaped backtick", GetSingleRefId(result));
        }

        // ============================================================================
        // Single Quote Tests (with escaping)
        // ============================================================================

        [Fact]
        public static void TestSingleQuoteWithEscapedSingleQuote()
        {
            var parser = new Parser();
            var result = parser.Parse("'text with '' escaped quote'");
            Assert.Equal("text with ' escaped quote", GetSingleRefId(result));
        }

        // ============================================================================
        // Double Quote Tests (with escaping)
        // ============================================================================

        [Fact]
        public static void TestDoubleQuoteWithEscapedDoubleQuote()
        {
            var parser = new Parser();
            var result = parser.Parse("\"text with \"\" escaped quote\"");
            Assert.Equal("text with \" escaped quote", GetSingleRefId(result));
        }

        // ============================================================================
        // Double Quotes (2 quote chars) Tests
        // ============================================================================

        [Fact]
        public static void TestDoubleDoubleQuotes()
        {
            var parser = new Parser();
            var result = parser.Parse("\"\"double double quotes\"\"");
            Assert.Equal("double double quotes", GetSingleRefId(result));
        }

        [Fact]
        public static void TestDoubleDoubleQuotesWithSingleQuoteInside()
        {
            var parser = new Parser();
            var result = parser.Parse("\"\"text with \" inside\"\"");
            Assert.Equal("text with \" inside", GetSingleRefId(result));
        }

        [Fact]
        public static void TestDoubleDoubleQuotesWithEscape()
        {
            var parser = new Parser();
            var result = parser.Parse("\"\"text with \"\"\"\" escaped double\"\"");
            Assert.Equal("text with \"\" escaped double", GetSingleRefId(result));
        }

        [Fact]
        public static void TestDoubleSingleQuotes()
        {
            var parser = new Parser();
            var result = parser.Parse("''double single quotes''");
            Assert.Equal("double single quotes", GetSingleRefId(result));
        }

        [Fact]
```

```
        public static void TestDoubleSingleQuotesWithSingleQuoteInside()
        {
            var parser = new Parser();
            var result = parser.Parse("'''text with ' inside'''");
            Assert.Equal("text with ' inside", GetSingleRefId(result));
        }

        [Fact]
        public static void TestDoubleSingleQuotesWithEscape()
        {
            var parser = new Parser();
            var result = parser.Parse("'''text with '''' escaped single'''");
            Assert.Equal("text with '' escaped single", GetSingleRefId(result));
        }

        [Fact]
        public static void TestDoubleBacktickQuotes()
        {
            var parser = new Parser();
            var result = parser.Parse("``double backtick quotes``");
            Assert.Equal("double backtick quotes", GetSingleRefId(result));
        }

        [Fact]
        public static void TestDoubleBacktickQuotesWithBacktickInside()
        {
            var parser = new Parser();
            var result = parser.Parse("``text with ` inside``");
            Assert.Equal("text with ` inside", GetSingleRefId(result));
        }

        [Fact]
        public static void TestDoubleBacktickQuotesWithEscape()
        {
            var parser = new Parser();
            var result = parser.Parse("``text with ```` escaped backtick``");
            Assert.Equal("text with `` escaped backtick", GetSingleRefId(result));
        }

        // ============================================================================
        // Triple Quotes (3 quote chars) Tests
        // ============================================================================

        [Fact]
        public static void TestTripleDoubleQuotes()
        {
            var parser = new Parser();
            var result = parser.Parse("\"\"\"triple double quotes\"\"\"");
            Assert.Equal("triple double quotes", GetSingleRefId(result));
        }

        [Fact]
        public static void TestTripleDoubleQuotesWithDoubleQuoteInside()
        {
            var parser = new Parser();
            var result = parser.Parse("\"\"\"text with \"\" inside\"\"\"");
            Assert.Equal("text with \"\" inside", GetSingleRefId(result));
        }

        [Fact]
        public static void TestTripleDoubleQuotesWithEscape()
        {
            var parser = new Parser();
            var result = parser.Parse("\"\"\"text with \"\"\"\"\"\" escaped triple\"\"\"");
            Assert.Equal("text with \"\"\" escaped triple", GetSingleRefId(result));
        }

        [Fact]
        public static void TestTripleSingleQuotes()
        {
            var parser = new Parser();
            var result = parser.Parse("'''triple single quotes'''");
            Assert.Equal("triple single quotes", GetSingleRefId(result));
        }

        [Fact]
        public static void TestTripleBacktickQuotes()
        {
```

```
        var parser = new Parser();
        var result = parser.Parse("```triple backtick quotes```");
        Assert.Equal("triple backtick quotes", GetSingleRefId(result));
    }

    // ============================================================================
    // Quadruple Quotes (4 quote chars) Tests
    // ============================================================================

    [Fact]
    public static void TestQuadrupleDoubleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"\"\"quadruple double quotes\"\"\"\"");
        Assert.Equal("quadruple double quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestQuadrupleSingleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("''''quadruple single quotes'''''");
        Assert.Equal("quadruple single quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestQuadrupleBacktickQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("`` ``quadruple backtick quotes`` ``");
        Assert.Equal("quadruple backtick quotes", GetSingleRefId(result));
    }

    // ============================================================================
    // Quintuple Quotes (5 quote chars) Tests
    // ============================================================================

    [Fact]
    public static void TestQuintupleDoubleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"\"\"\"quintuple double quotes\"\"\"\"\"");
        Assert.Equal("quintuple double quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestQuintupleSingleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("'''''quintuple single quotes''''''");
        Assert.Equal("quintuple single quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestQuintupleBacktickQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("`` ``quintuple backtick quotes`` ``");
        Assert.Equal("quintuple backtick quotes", GetSingleRefId(result));
    }

    // ============================================================================
    // Complex Scenarios Tests
    // ============================================================================

    [Fact]
    public static void TestMixedQuotesInLink()
    {
        var parser = new Parser();
        var result = parser.Parse("(\"double\" 'single' `backtick`)");
        Assert.Single(result);
        Assert.NotNull(result[0].Values);
        Assert.Equal(3, result[0].Values!.Count);
        Assert.Equal("double", result[0].Values[0].Id);
        Assert.Equal("single", result[0].Values[1].Id);
        Assert.Equal("backtick", result[0].Values[2].Id);
    }

    [Fact]
```

```
        public static void TestBacktickAsIdInLink()
        {
            var parser = new Parser();
            var result = parser.Parse("(`myId`: value1 value2)");
            Assert.Single(result);
            Assert.Equal("myId", result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Equal(2, result[0].Values!.Count);
        }

        [Fact]
        public static void TestCodeBlockLikeContent()
        {
            var parser = new Parser();
            var result = parser.Parse("```const x = 1;```");
            Assert.Equal("const x = 1;", GetSingleRefId(result));
        }

        [Fact]
        public static void TestNestedQuotesInMarkdown()
        {
            var parser = new Parser();
            var result = parser.Parse("``Use `code` in markdown``");
            Assert.Equal("Use `code` in markdown", GetSingleRefId(result));
        }

        [Fact]
        public static void TestJsonStringWithQuotes()
        {
            var parser = new Parser();
            var result = parser.Parse("\"\"{ \"key\": \"value\"}\"\"");
            Assert.Equal("{ \"key\": \"value\"}", GetSingleRefId(result));
        }

        // ============================================================================
        // Edge Cases
        // ============================================================================

        [Fact]
        public static void TestWhitespacePreservedInQuotes()
        {
            var parser = new Parser();
            var result = parser.Parse("\"  spaces  \"");
            Assert.Equal("  spaces  ", GetSingleRefId(result));
        }

        [Fact]
        public static void TestMultilineInDoubleDoubleQuotes()
        {
            var parser = new Parser();
            var result = parser.Parse("(\"\"line1\nline2\"\")");
            Assert.Single(result);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("line1\nline2", result[0].Values![0].Id);
        }

        // ============================================================================
        // Unlimited Quotes (6+ quote chars) Tests
        // ============================================================================

        [Fact]
        public static void TestUnlimitedQuotes6()
        {
            // Test 6-quote strings
            var parser = new Parser();
            var result = parser.Parse("\"\"\"\"\"\"hello\"\"\"\"\"\"");
            Assert.Equal("hello", GetSingleRefId(result));
        }

        [Fact]
        public static void TestUnlimitedQuotes10()
        {
            // Test 10-quote strings
            var parser = new Parser();
            var result = parser.Parse("\"\"\"\"\"\"\"\"\"\"very deeply quoted\"\"\"\"\"\"\"\"\"\"");
            Assert.Equal("very deeply quoted", GetSingleRefId(result));
```

```
        }

        [Fact]
        public static void TestUnlimitedQuotes6WithInnerQuotes()
        {
            // Test 6-quote strings with inner 5-quote sequences
            var parser = new Parser();
            var result = parser.Parse("\"\"\"\"\"\"hello with \"\"\"\"\" five quotes inside\"\"\"\"\"\"");
            Assert.Equal("hello with \"\"\"\"\" five quotes inside", GetSingleRefId(result));
        }

        [Fact]
        public static void TestUnlimitedSingleQuotes7()
        {
            // Test 7-quote single quote strings
            var parser = new Parser();
            var result = parser.Parse("'''''''seven single quotes'''''''");
            Assert.Equal("seven single quotes", GetSingleRefId(result));
        }

        [Fact]
        public static void TestUnlimitedBackticks8()
        {
            // Test 8-quote backtick strings
            var parser = new Parser();
            var result = parser.Parse("`eight backticks`");
            Assert.Equal("eight backticks", GetSingleRefId(result));
        }
    }
}
```

1.19 ./Link.Foundation.Links.Notation.Tests/MultilineParserTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class MultilineParserTests
    {
        [Fact]
        public static void TwoLinksTest()
        {
            var source = @"(first: x y)
(second: a b)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void ParseAndStringifyTest()
        {
            var source = @"(papa (lovesMama: loves mama))
(son lovesMama)
(daughter lovesMama)
(all (love mama))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void ParseAndStringifyTest2()
        {
            var source = @"father (lovesMom: loves mom)
son lovesMom
daughter lovesMom
all (love mom)";
            var links = new Parser().Parse(source);
            var target = links.Format(lessParentheses: true);
            Assert.Equal(source, target);
        }

        [Fact]
        public static void ParseAndStringifyWithLessParenthesesTest()
        {
```

```
            var source = @"lovesMama: loves mama
papa lovesMama
son lovesMama
daughter lovesMama
all (love mama)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format(lessParentheses: true);
            Assert.Equal(source, target);
        }

        [Fact]
        public static void DuplicateIdentifiersTest()
        {
            var source = @"(a: a b)
(a: b c)";
            var target = @"(a: a b)
(a: b c)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void ComplexStructureTest()
        {
            var input = @"(Type: Type Type)
  Number
  String
  Array
  Value
    (property: name type)
    (method: name params return)";

            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MixedFormatsTest()
        {
            // Mix of single-line and multi-line formats
            var input = @"id1: value1
(id2: value2 value3)
simple ref
(complex:
  nested1
  nested2
)";

            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MultilineWithIdTest()
        {
            // Test multi-line link with id
            var input = "(id: value1 value2)";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MultipleTopLevelElementsTest()
        {
            // Test multiple top-level elements
            var input = "(elem1: val1)\n(elem2: val2)";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MultilineSimpleLinksTest()
        {
            var input = "(1: 1 1)\n(2: 2 2)";
            var parser = new Parser();
            var parsed = parser.Parse(input);
```

```
            Assert.NotEmpty(parsed);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.Contains("(1: 1 1)", output);
            Assert.Contains("(2: 2 2)", output);

            // Validate alternate formatting matches input
            var outputAlternate = parsed.Format();
            Assert.Equal(input, outputAlternate);
        }

        [Fact]
        public static void IndentedChildrenTest()
        {
            var input = "parent\n  child1\n  child2";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // The parsed structure should have parent with children
            Assert.NotEmpty(parsed);
        }
    }
}
```

1.20 ./Link.Foundation.Links.Notation.Tests/MultilineQuotedStringTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class MultilineQuotedStringTests
    {
        [Fact]
        public static void TestMultilineDoubleQuotedReference()
        {
            var input = @"(
  ""long
string literal representing
the reference""

  'another
long string literal
as another reference'
)";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);

            var link = result[0];
            Assert.Null(link.Id);
            Assert.NotNull(link.Values);
            Assert.Equal(2, link.Values.Count);

            Assert.Equal(@"long
string literal representing
the reference", link.Values[0].Id);

            Assert.Equal(@"another
long string literal
as another reference", link.Values[1].Id);
        }

        [Fact]
        public static void TestSimpleMultilineDoubleQuoted()
        {
            var input = @""line1
line2"")";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);

            var link = result[0];
            Assert.Null(link.Id);
            Assert.NotNull(link.Values);
            Assert.Single(link.Values);
```

```
            Assert.Equal("line1\nline2", link.Values[0].Id);
        }

        [Fact]
        public static void TestSimpleMultilineSingleQuoted()
        {
            var input = @"('line1
line2')";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);

            var link = result[0];
            Assert.Null(link.Id);
            Assert.NotNull(link.Values);
            Assert.Single(link.Values);
            Assert.Equal("line1\nline2", link.Values[0].Id);
        }

        [Fact]
        public static void TestMultilineQuotedAsId()
        {
            var input = @""multi
line
id"": value1 value2)";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);

            var link = result[0];
            Assert.Equal("multi\nline\nid", link.Id);
            Assert.NotNull(link.Values);
            Assert.Equal(2, link.Values.Count);
        }
    }
}
```

1.21 ./Link.Foundation.Links.Notation.Tests/NestedParserTests.cs

```
using System;
using System.Linq;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class NestedParserTests
    {
        [Fact]
        public static void SignificantWhitespaceTest()
        {
            var source = @"
users
    user1
        id
            43
        name
            first
                John
            last
                Williams
        location
            New York
        age
            23
    user2
        id
            56
        name
            first
                Igor
            middle
                Petrovich
            last
                Ivanov
        location
            Moscow
        age
            20";
```

```
            var target = @"(users)
((users) (user1))
(((users) (user1)) (id))
((((users) (user1)) (id)) (43))
(((users) (user1)) (name))
((((users) (user1)) (name)) (first))
(((((users) (user1)) (name)) (first)) (John))
((((users) (user1)) (name)) (last))
(((((users) (user1)) (name)) (last)) (Williams))
(((users) (user1)) (location))
((((users) (user1)) (location)) (New York))
(((users) (user1)) (age))
((((users) (user1)) (age)) (23))
((users) (user2))
(((users) (user2)) (id))
((((users) (user2)) (id)) (56))
(((users) (user2)) (name))
((((users) (user2)) (name)) (first))
(((((users) (user2)) (name)) (first)) (Igor))
((((users) (user2)) (name)) (middle))
(((((users) (user2)) (name)) (middle)) (Petrovich))
((((users) (user2)) (name)) (last))
(((((users) (user2)) (name)) (last)) (Ivanov))
(((users) (user2)) (location))
((((users) (user2)) (location)) (Moscow))
(((users) (user2)) (age))
((((users) (user2)) (age)) (20))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void SimpleSignificantWhitespaceTest()
        {
            var source = @"a
    b
    c";
            var target = @"(a)
((a) (b))
((a) (c))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void TwoSpacesSizedWhitespaceTest()
        {
            var source = @"
users
  user1";
            var target = @"(users)
((users) (user1))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void ParseNestedStructureWithIndentationTest()
        {
            var source = @"parent
  child1
  child2";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            // Should create 3 links: (parent), (parent child1), (parent child2)
            Assert.Equal(3, links.Count);
        }

        [Fact]
        public static void IndentationConsistencyTest()
        {
            // Test that indentation must be consistent
```

```
            var input = @"parent
  child1
   child2"; // Inconsistent indentation
            var result = new Parser().Parse(input);
            // This should parse but child2 won't be a child of parent due to different indentation
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void IndentationBasedChildrenTest()
        {
            var input = @"parent
  child1
  child2
    grandchild";
            var parser = new Parser();
            var result = parser.Parse(input);
            Assert.NotNull(result);
            // Expected flattened links:
            // (parent), (parent child1), (parent child2), ((parent child2) grandchild)
            Assert.Equal(4, result.Count);
        }

        [Fact]
        public static void ComplexIndentationTest()
        {
            var input = @"root
  level1a
    level2a
    level2b
  level1b
    level2c";
            var parser = new Parser();
            var result = parser.Parse(input);
            Assert.NotNull(result);
            // Expected flattened links:
            // (root), (root level1a), ((root level1a) level2a), ((root level1a) level2b),
            // (root level1b), ((root level1b) level2c)
            Assert.Equal(6, result.Count);
        }

        [Fact]
        public static void NestedLinksTest()
        {
            var input = "(1: (2: (3: 3)))";
            var parser = new Parser();
            var parsed = parser.Parse(input);
            Assert.NotEmpty(parsed);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.NotEmpty(output);

            // Validate that the structure is properly nested
            Assert.Single(parsed);
        }

        [Fact]
        public static void IndentationParserTest()
        {
            var input = "parent\n  child1\n  child2";
            var parser = new Parser();
            var result = parser.Parse(input);
            Assert.NotEmpty(result);
            // Should have parent link
            var parentLink = result.FirstOrDefault(l => l.Id == "parent");
        }

        [Fact]
        public static void NestedIndentationParserTest()
        {
            var input = "parent\n  child\n    grandchild";
            var parser = new Parser();
            var result = parser.Parse(input);
            Assert.NotEmpty(result);
            // Should create nested structure with proper hierarchy
            Assert.True(result.Count >= 1);
        }
```

```

        [Fact]
        public static void ThreeLevelNestingRoundtripTest()
        {
            var input = "(1: (2: (3: 3)))";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate round-trip
            var output = parsed.Format();
            Assert.Equal(input, output);
        }

        [Fact]
        public static void DeepNestedStructureRoundtripTest()
        {
            var input = "(a: (b: (c: (d: d))))";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate round-trip
            var output = parsed.Format();
            Assert.Equal(input, output);
        }

        [Fact]
        public static void MultipleNestedLinksRoundtripTest()
        {
            var input = "(parent: (child1: value1) (child2: value2))";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate round-trip
            var output = parsed.Format();
            Assert.Equal(input, output);
        }
    }
}
```

1.22 ./Link.Foundation.Links.Notation.Tests/NestedSelfReferenceTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class NestedSelfReferenceTests
    {
        [Fact]
        public static void NestedSelfReferencedObjectInPairValueTest()
        {
            // Test case from PARSER BUG.md
            // This should parse a dict with two pairs, where the second pair's value
            // is itself a self-referenced dict definition (obj 1: dict ...)
            var notation = "(obj 0: dict ((str bmFtZQ==) (str ZGljdDE=)) ((str b3RoZXI=) (obj 1:
↪ dict ((str bmFtZQ==) (str ZGljdDI=)) ((str b3RoZXI=) obj 0))))";

            var parser = new Parser();
            var links = parser.Parse(notation);

            // Should parse exactly one top-level link
            Assert.Single(links);

            var link = links[0];

            // Top-level link should have ID "obj 0"
            Assert.Equal("obj 0", link.Id);

            // Should have: type marker + 2 pairs = 3 values
            Assert.Equal(3, link.Values.Count);

            // First value is the type marker "dict"
            Assert.Equal("dict", link.Values[0].Id);

            // Second and third values are the two pairs
            var pair1 = link.Values[1];
            var pair2 = link.Values[2];

            // Pair 1: ((str bmFtZQ==) (str ZGljdDE=))
            // This is a parenthesized expression containing two sub-expressions
```

```
            Assert.Null(pair1.Id);
            Assert.Equal(2, pair1.Values.Count);

            // First element of pair1: (str bmFtZQ==)
            Assert.Null(pair1.Values[0].Id);
            Assert.Equal(2, pair1.Values[0].Values.Count);
            Assert.Equal("str", pair1.Values[0].Values[0].Id);
            Assert.Equal("bmFtZQ==", pair1.Values[0].Values[1].Id);

            // Second element of pair1: (str ZGljdDE=)
            Assert.Null(pair1.Values[1].Id);
            Assert.Equal(2, pair1.Values[1].Values.Count);
            Assert.Equal("str", pair1.Values[1].Values[0].Id);
            Assert.Equal("ZGljdDE=", pair1.Values[1].Values[1].Id);

            // Pair 2: ((str b3RoZXI=) (obj 1: dict ...))
            // This is a parenthesized expression containing two sub-expressions
            Assert.Null(pair2.Id);
            Assert.Equal(2, pair2.Values.Count);

            // First element of pair2: (str b3RoZXI=)
            Assert.Null(pair2.Values[0].Id);
            Assert.Equal(2, pair2.Values[0].Values.Count);
            Assert.Equal("str", pair2.Values[0].Values[0].Id);
            Assert.Equal("b3RoZXI=", pair2.Values[0].Values[1].Id);

            // Second element of pair2: (obj 1: dict ((str bmFtZQ==) (str ZGljdDI=)) ((str b3RoZXI=) obj 0))
            // THIS IS THE KEY TEST - obj 1 should have its ID preserved
            var obj1 = pair2.Values[1];
            Assert.Equal("obj 1", obj1.Id);
            Assert.NotNull(obj1.Values);
            Assert.Equal(3, obj1.Values.Count); // type marker + 2 pairs

            // obj 1's type marker
            Assert.Equal("dict", obj1.Values[0].Id);

            // obj 1's first pair: ((str bmFtZQ==) (str ZGljdDI=))
            var obj1Pair1 = obj1.Values[1];
            Assert.Equal(2, obj1Pair1.Values.Count);
            Assert.Null(obj1Pair1.Values[0].Id);
            Assert.Equal(2, obj1Pair1.Values[0].Values.Count);
            Assert.Equal("str", obj1Pair1.Values[0].Values[0].Id);
            Assert.Equal("bmFtZQ==", obj1Pair1.Values[0].Values[1].Id);
            Assert.Null(obj1Pair1.Values[1].Id);
            Assert.Equal(2, obj1Pair1.Values[1].Values.Count);
            Assert.Equal("str", obj1Pair1.Values[1].Values[0].Id);
            Assert.Equal("ZGljdDI=", obj1Pair1.Values[1].Values[1].Id);

            // obj 1's second pair: ((str b3RoZXI=) obj 0) - reference back to obj 0
            var obj1Pair2 = obj1.Values[2];
            Assert.Equal(2, obj1Pair2.Values.Count);
            Assert.Null(obj1Pair2.Values[0].Id);
            Assert.Equal(2, obj1Pair2.Values[0].Values.Count);
            Assert.Equal("str", obj1Pair2.Values[0].Values[0].Id);
            Assert.Equal("b3RoZXI=", obj1Pair2.Values[0].Values[1].Id);
            Assert.Equal("obj 0", obj1Pair2.Values[1].Id);
            Assert.Null(obj1Pair2.Values[1].Values); // Just a reference, no nested values
        }

        [Fact]
        public static void SelfReferenceAsDirectChildWorksCorrectlyTest()
        {
            // This pattern should work (and did work before)
            var notation = "(obj 0: list (int 1) (int 2) (obj 1: list (int 3) (int 4) obj 0))";

            var parser = new Parser();
            var links = parser.Parse(notation);

            Assert.Single(links);
            Assert.Equal("obj 0", links[0].Id);
            Assert.Equal(4, links[0].Values.Count); // list + 1 + 2 + obj 1

            // The fourth value should be obj 1 with a self-reference
            var obj1 = links[0].Values[3];
            Assert.Equal("obj 1", obj1.Id);
            Assert.Equal(4, obj1.Values.Count); // list + 3 + 4 + obj 0
            Assert.Equal("obj 0", obj1.Values[3].Id); // Reference to obj 0
        }
```

```
    }
}
```

1.23 ./Link.Foundation.Links.Notation.Tests/SingleLineParserTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class SingleLineParserTests
    {
        [Fact]
        public static void SingleLinkTest()
        {
            var source = @"(address: source target)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void TripletSingleLinkTest()
        {
            var source = @"(papa has car)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void BugTest1()
        {
            var source = @"(ignore conan-center-index repository)";
            var links = new Parser().Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void QuotedReferencesTest()
        {
            var source = @"(a: 'b' ""c"")";
            var target = @"(a: b c)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void QuotedReferencesWithSpacesTest()
        {
            var source = @"('a a': 'b b' ""c c"")";
            var target = @"('a a': 'b b' 'c c')";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void ParseSimpleReferenceTest()
        {
            var source = "test";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            Assert.Single(links);
            // Simple reference creates a singlet link with null Id and one value
            Assert.Null(links[0].Id);
            Assert.NotNull(links[0].Values);
            Assert.Single(links[0].Values);
            Assert.Equal("test", links[0].Values?[0].Id);
            Assert.Null(links[0].Values?[0].Values);
        }

        [Fact]
```

```
        public static void ParseReferenceWithColonAndValuesTest()
        {
            var source = "parent: child1 child2";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            Assert.Single(links);
            Assert.Equal("parent", links[0].Id);
            Assert.NotNull(links[0].Values);
            Assert.Equal(2, links[0].Values!.Count);
            Assert.Equal("child1", links[0].Values![0].Id);
            Assert.Equal("child2", links[0].Values![1].Id);
        }

        [Fact]
        public static void ParseMultilineLinkTest()
        {
            var source = "(parent: child1 child2)";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            Assert.Single(links);
            Assert.Equal("parent", links[0].Id);
            Assert.NotNull(links[0].Values);
            Assert.Equal(2, links[0].Values!.Count);
        }

        [Fact]
        public static void ParseValuesOnlyTest()
        {
            var source = ": value1 value2";
            var parser = new Parser();
            // Standalone ':' is now forbidden and should throw an exception
            Assert.Throws<FormatException>(() => parser.Parse(source));
        }

        [Fact]
        public static void SingletLinkTest()
        {
            // Test singlet link
            var input = "(singlet)";
            var result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("singlet", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);
        }

        [Fact]
        public static void ValueLinkTest()
        {
            // Test value link
            var input = "(value1 value2 value3)";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void QuotedReferencesWithSpecialCharsTest()
        {
            // Test quoted references
            var input = @(""id with spaces"": ""value with spaces"")";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void SingleQuotedReferencesTest()
        {
            // Test single-quoted references
            var input = "('id': 'value')";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
```

```
        public static void ParseQuotedReferencesValuesOnlyTest()
        {
            var source = "\"has space\" 'has:colon'";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            Assert.Single(links);
            Assert.Null(links[0].Id);
            Assert.NotNull(links[0].Values);
            Assert.Equal(2, links[0].Values!.Count);
            Assert.Equal("has space", links[0].Values![0].Id);
            Assert.Equal("has:colon", links[0].Values![1].Id);
            var formatted = links.Format();
            Assert.Equal("('has space' 'has:colon')", formatted);
        }

        [Fact]
        public static void NestedLinksTest()
        {
            // Test nested links
            var input = "(outer: (inner: value))";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void HyphenatedIdentifiersTest()
        {
            // Test support for hyphenated identifiers like in BugTest1
            var source = @"(conan-center-index: repository info)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void MultipleWordsInQuotesTest()
        {
            var source = @""("New York"": city state)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            // Should preserve quotes for multi-word references
            Assert.Contains("New York", target);
        }

        [Fact]
        public static void SpecialCharactersInQuotesTest()
        {
            var input = @""("key:with:colons"": ""value(with)parens"")";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            input = @"('key with spaces': 'value: with special chars')";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void DeeplyNestedTest()
        {
            var input = "(a: (b: (c: (d: (e: value)))))";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void SingleLineWithIdTest()
        {
            // Test single-line link with id
            var input = "id: value1 value2";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void SingleLineWithoutIdTest()
```

```
        {
            // Test link without id (single-line) - now forbidden
            var input = ": value1 value2";
            Assert.Throws<FormatException>(() => new Parser().Parse(input));
        }

        [Fact]
        public static void MultilineWithoutIdTest()
        {
            // Test link without id (multi-line) - now forbidden
            var input = "(: value1 value2)";
            Assert.Throws<FormatException>(() => new Parser().Parse(input));
        }

        [Fact]
        public static void SimpleRefTest()
        {
            var input = "simple ref";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MultiLineLinkWithIdTest()
        {
            var input = "(id: value1 value2)";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void LinkWithoutIdMultiLineTest()
        {
            var input = "(: value1 value2)";
            Assert.Throws<FormatException>(() => new Parser().Parse(input));
        }

        [Fact]
        public static void SimpleReferenceParserTest()
        {
            var input = "hello";
            var result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("hello", result[0].Values?[0].Id);
        }

        [Fact]
        public static void QuotedReferenceParserTest()
        {
            var input = "\"hello world\"";
            var result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("hello world", result[0].Values?[0].Id);
        }

        [Fact]
        public static void SingletLinkParserTest()
        {
            var input = "(singlet)";
            var result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("singlet", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);
        }

        [Fact]
        public static void ValueLinkParserTest()
        {
            var input = "(a b c)";
            var result = new Parser().Parse(input);
```

```
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.Equal(3, result[0].Values?.Count);
        }

        [Fact]
        public static void ParseQuotedReferencesTest()
        {
            var input = "(\"quoted id\": \"value with spaces\")";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("quoted id", formatted);
            Assert.Contains("value with spaces", formatted);
        }

        [Fact]
        public static void LinkWithoutIdSingleLineTest()
        {
            // Test that standalone ':' is forbidden and should throw
            var input = ": value1 value2";
            var parser = new Parser();

            // C# parser forbids this syntax (like JS/Rust)
            Assert.Throws<FormatException>(() => parser.Parse(input));
        }

        [Fact]
        public static void SingleLineLinkWithIdTest()
        {
            var input = "id: value1 value2";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.NotNull(result[0].Id);
            Assert.Equal("id", result[0].Id);
        }

        [Fact]
        public static void LinkWithIdTest()
        {
            var input = "(id: a b c)";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Equal("id", result[0].Id);
            Assert.Equal(3, result[0].Values?.Count);
        }

        [Fact]
        public static void QuotedReferenceTest()
        {
            var input = "\"quoted value\"";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values);
            Assert.Equal("quoted value", result[0].Values[0].Id);
        }

        [Fact]
        public static void SimpleReferenceTest()
        {
            var input = "simplereference";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
        }

        [Fact]
        public static void SingleLineLinkTest()
        {
```

```
            var input = "myid: value1 value2";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Equal("myid", result[0].Id);
            Assert.Equal(2, result[0].Values?.Count);
        }

        [Fact]
        public static void ValuesOnlyInParenthesesTest()
        {
            var input = "(value1 value2)";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.Equal(2, result[0].Values?.Count);
        }

        [Fact]
        public static void ParseValuesOnlyStandaloneColonTest()
        {
            // Test that standalone ':' is forbidden and should throw
            var input = ": value1 value2";
            var parser = new Parser();

            // C# parser forbids this syntax (like JS/Rust)
            Assert.Throws<FormatException>(() => parser.Parse(input));
        }

        [Fact]
        public static void QuotedReferencesWithSpacesInLinkTest()
        {
            var input = "(id: \"value with spaces\")";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Equal("id", result[0].Id);
            Assert.Single(result[0].Values);
            Assert.Equal("value with spaces", result[0].Values?[0].Id);
        }
    }
}
```

1.24 ./Link.Foundation.Links.Notation.Tests/TupleTests.cs

```
using System.Collections.Generic;
using Xunit;
using LinkType = Link.Foundation.Links.Notation.Link<string>;
using id = Link.Foundation.Links.Notation.id<string>;
using   = Link.Foundation.Links.Notation. <string>;

namespace Link.Foundation.Links.Notation.Tests
{
    public class TupleTests
    {
        [Fact]
        public void TupleToLinkTest()
        {
            var source = @"(papa (lovesMama: loves mama))
(son lovesMama)
(daughter lovesMama)
(all (love mama))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var targetFromString = links.Format();

            IList<LinkType> constructedLinks = new List<LinkType>()
            {
                ("papa", ( )("lovesMama", "loves", "mama")),
                ("son", "lovesMama"),
                ("daughter", "lovesMama"),
                ("all", ("love", "mama")),
            };
            var targetFromTuples = constructedLinks.Format();
            Assert.Equal(targetFromString, targetFromTuples);
        }
```

```

        [Fact]
        public void NamedTupleToLinkTest()
        {
            var source = @"(papa (lovesMama: loves mama))
(son lovesMama)
(daughter lovesMama)
(all (love mama))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var targetFromString = links.Format();

            IList<LinkType> constructedLinks = new List<LinkType>()
            {
                ("papa", ((id)"lovesMama", "loves", "mama")),
                ("son", "lovesMama"),
                ("daughter", "lovesMama"),
                ("all", ("love", "mama")),
            };
            var targetFromTuples = constructedLinks.Format();
            Assert.Equal(targetFromString, targetFromTuples);
        }
    }
}
```

Index

./Link.Foundation.Links.Notation.Tests/ApiTests.cs, 15
./Link.Foundation.Links.Notation.Tests/EdgeCaseParserTests.cs, 17
./Link.Foundation.Links.Notation.Tests/FormatConfigTests.cs, 20
./Link.Foundation.Links.Notation.Tests/IndentationConsistencyTests.cs, 21
./Link.Foundation.Links.Notation.Tests/IndentedIdSyntaxTests.cs, 23
./Link.Foundation.Links.Notation.Tests/LinkTests.cs, 25
./Link.Foundation.Links.Notation.Tests/LinksGroupTests.cs, 27
./Link.Foundation.Links.Notation.Tests/MixedIndentationModesTests.cs, 28
./Link.Foundation.Links.Notation.Tests/MultiQuoteParserTests.cs, 31
./Link.Foundation.Links.Notation.Tests/MultilineParserTests.cs, 36
./Link.Foundation.Links.Notation.Tests/MultilineQuotedStringTests.cs, 38
./Link.Foundation.Links.Notation.Tests/NestedParserTests.cs, 39
./Link.Foundation.Links.Notation.Tests/NestedSelfReferenceTests.cs, 42
./Link.Foundation.Links.Notation.Tests/SingleLineParserTests.cs, 44
./Link.Foundation.Links.Notation.Tests/TupleTests.cs, 49
./Link.Foundation.Links.Notation/FormatConfig.cs, 1
./Link.Foundation.Links.Notation/FormatOptions.cs, 1
./Link.Foundation.Links.Notation/ILinksGroupListExtensions.cs, 2
./Link.Foundation.Links.Notation/IListExtensions.cs, 3
./Link.Foundation.Links.Notation/Link.cs, 5
./Link.Foundation.Links.Notation/LinkFormatExtensions.cs, 9
./Link.Foundation.Links.Notation/LinksGroup.cs, 11
./Link.Foundation.Links.Notation/ .cs, 13
./Link.Foundation.Links.Notation/id.cs, 14